

Analyse d'algorithmes - Travail Pratique

Étude d'un problème NP-complet : 3-Coloration

Organisation

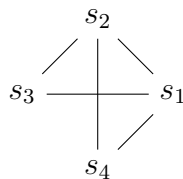
- Ce TP est à réaliser seul ou en binôme.
- Deux séances de 3h30 y sont dédiées.

Les entrées des programmes seront des graphes, des CNFs et des colorations. Dans les deux premiers cas, je vous fournis des fichiers qui génèrent des instances aléatoires. L'entrée sera donc donnée par des fichiers textes encodant ces instances.

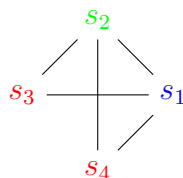
Problème 3-COLORATION :

Dans le cadre de la théorie des graphes, un graphe est 3-coloriable si on peut colorier tous ses sommets avec seulement trois couleurs de façon à ce que deux sommets voisins ne soient jamais de la même couleur.

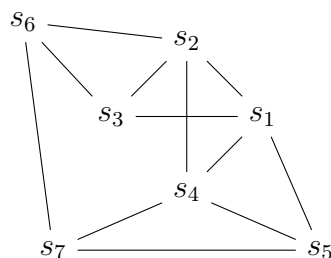
Le graphe



est 3-coloriable :



Alors que le graphe



n'est pas 3-coloriable (essayer un peu de le 3-colorer pour vous en convaincre). Le problème 3-coloration (nommé 3Col) est : donné un graphe en entrée, peut-il être colorié avec seulement trois couleurs?

1 2-Coloration

On commence par un cas simplifié de notre problème. Les graphes 3-colorables sont plus souvent appelés bipartis. On peut facilement tester si un graphe est biparti en partant d'un sommet (dans chaque composante connexe) et essayant de colorer ses voisins. S'il y a une incompatibilité, alors il n'existe pas de 2-coloriage.

- Écrire un programme qui teste si un graphe est biparti.

Entrée Un graphe.

Sortie Accepter si le graphe est biparti.

- Quelle est la complexité de l'algorithme ? Le temps de calcul correspond à la complexité attendue ?

2 3 - Coloration

1. On a vu en cours qu'un problème est dans NP si l'on peut tester si une solution est correcte en temps polynomial. Ici, la solution est juste une "3-coloration du graphe" (par exemple un dico qui à chaque sommet associe une couleur).

Entrée Un graphe et une coloration de chaque sommet du graphe.

Sortie Accepter si la coloration est une 3-coloration valide.

Cette fonction doit posséder une complexité temporelle polynomiale en fonction de la taille de l'instance.

2. On cherche maintenant à coder le problème 3Col.

Le principe du backtracking est de construire graduellement une solution en faisant des choix plus ou moins arbitraires et en changeant ces choix lorsqu'on ne peut plus progresser ("backtrack"). Plus précisément, on essaie de créer une solution de 3Col $(x_1, x_2, \dots, x_n)$. Un algorithme de type *backtracking* va attribuer une valeur (i.e., une couleur) à x_1 , puis une valeur à x_2 , puis à x_3 et ainsi de suite. À chaque étape on teste si la solution partielle $(x_1, x_2, \dots, x_i)$ contient une contradiction. Si c'est le cas, on change la valeur de x_i et on teste à nouveau. Si toutes les valeurs possibles de x_i ont été testées et qu'aucune n'est possible, alors on remonte à x_{i-1} et lui attribue une autre valeur. L'algorithme accepte lorsque x_n s'est vu attribué une valeur et que toutes les valeurs $(x_1, \dots, x_n)$ sont cohérentes. Si toutes les entrées ont été testées et qu'aucune ne produit une solution, on peut donc affirmer qu'il n'existe pas de solution.

Entrée Un graphe donné par un fichier externe.

Sortie Le programme résout le problème 3Col via backtracking.

3. Quelle est la complexité de ce programme ?

3 SAT-solver et réductions

On va utiliser le SAT-solver pycosat.

pip install pycosat

Pour ajouter des contraintes de comptage, utiliser la librairie pysat

pip install python-sat

Pour information l'entrée de la fonction solve de pycosat attend une liste de contraintes. Chaque contrainte sera elle-même une liste de nombres (littéral positif si nombre positif et littéral négatif si nombre négatif). Par exemple la contrainte [3, -2, 1, 4] correspond à la contrainte $X_3 \vee (\neg X_2) \vee X_1 \vee X_4$.

Par exemple, pour encoder le problème de Clique(G , taille_clique), il suffit d'ajouter :

- des contraintes assurant que le nombre de variables positives doit être exactement taille_clique (ces contraintes sont créées par la fonction CardEnc.equals de pysat),
- pour chaque paire de sommets (u, v) sans arête, la contrainte $[-u, -v]$.

Attention, 0 ne doit pas être un indice de variables! Sinon, on ne pourrait pas différencier si c'est X_0 ou $\neg X_0$.

3.1 Échauffement avec les problèmes Cliques/EnsemblesIndépendants

On a vu en cours le problème Clique : Donné en entrée un graphe G et un entier k , on veut savoir s'il existe une clique de taille k dans G (c-à-d, un sous-graphe de G de taille k tel qu'entre chaque paire de ses sommets, il y ait toujours une arête).

On a vu aussi le problème EnsInd : Donné en entrée un graphe G et un entier k , on veut savoir s'il existe un ensemble indépendant de taille k dans G (c-à-d, un sous-graphe de G de taille k tel qu'entre chaque paire de ses sommets, il n'y ait jamais d'arête).

3.2 Un soupçon de SAT-solver

Je vous donne le code montré en cours pour résoudre le problème Clique avec le SAT-solver pycosat.

1. Écrire une version pour résoudre EnsInd toujours en utilisant le SAT-solver (normalement, il y a besoin de faire très peu de changements par rapport au fichier pour Clique).

3.3 Une pincée de réduction

On peut remarquer que si un graphe est 3-colorié, alors l'ensemble des sommets d'une même couleur correspond à un ensemble indépendant (c-à-d, un ensemble de sommets qui ne contient aucune arête entre eux).

Le problème 3CouvertureParCliques est : Donné un graphe en entier peut-on trouver trois cliques qui recouvrent l'ensemble des sommets.

1. Écrire un programme pour résoudre le problème 3CouvertureParCliques à l'aide d'un programme résolvant 3Col.
2. En déduire que si 3Col est NP-complet, alors 3CouvertureParCliques est NP-complet.

3.4 Retour au problème 3-Coloration

On a vu dans la deuxième section que l'on pouvait vérifier une solution pour 3Col en temps polynomial. Un tel problème est dans la classe de complexité NP. D'après le cours un tel problème peut être résolu par un SAT-solver.

1. Écrire un programme pour résoudre le problème 3Coloration à l'aide d'un SAT-solver.

Entrée Un graphe donné par un fichier externe.

Sortie Une coloration du graphe si possible.

S'aider du programme pour Clique.

2. Comparer cet algorithme avec celui de la section 2.

3.5 3-Coloration est NP-complet

Comme on a pu résoudre 3Col à l'aide d'un SAT-solver, cela signifie que ce problème est dans NP (= il est plus facile que SAT). En fait ce problème est aussi difficile que SAT (on dit qu'il est NP-complet). Pour le montrer, il suffit de montrer que 3SAT (équivalent à SAT) peut être résolu grâce à un appel à une fonction résolvant 3Col (des détails en-dessous).

1. Écrire un programme qui résout 3SAT à l'aide d'un programme résolvant 3Col.
2. En déduire que 3Col est NP-complet.

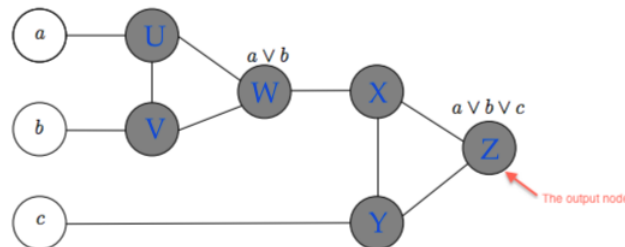
L'idée est de résoudre 3SAT en transformant la suite de contraintes L en un graphe $G(L)$ de façon à ce que toutes les contraintes de L soient satisfaisables si et seulement si le graphe $G(L)$ est 3-coloriable.

Dans la suite, on explique comment construire le graphe $G(L)$.

Pour faire celà, on va déjà se fixer des couleurs : commençons le graphe par un triangle de sommets nommés "V", "F" et "N" (quitte à renommer les couleurs à la fin, on pourra imaginer qu'un 3-coloriage de notre graphe coloriera "V" en vert (pour Vrai), "F" en fushia (pour Faux) et "N" en noir (pour Neutre)). Pour chaque variable x_i apparaissant dans L , on associera une paire de sommets Val_i et Opp_i (intuitivement le sommet Val_i sera de couleur vert si x_i est vraie et de couleur fushia si x_i est faux, et le contraire pour Opp_i). Ces deux sommets ne doivent donc pas être de couleur noir : on relie ces sommets à "N". Ils ne doivent pas être non plus de même couleur : on les relie donc entre eux.

Il faut rajouter maintenant les clauses (c-à-d les contraintes). Pour chaque contrainte l_a, l_b, l_c , on va rajouter 9 sommets ! On va déjà commencer par trois sommets (a,b, et c dans le graphique ci-dessous) qui correspondent aux trois littéraux l_a, l_b, l_c : comme pour les variables, ils devront être soit en vert soit en fushia en fonction de s'ils sont évalués à vrai ou faux. Donc, ils ne doivent pas être noir : on relie ces trois sommets à "N". Ensuite si l_a est positif (de la forme x_a), alors l_a doit être de la même couleur que le sommet Val_a : on le relie à Opp_a . De même, si l_a est négatif (de la forme $-x_a$), alors l_a doit être de la même couleur que le sommet Opp_a : on le relie à Val_a . Idem pour b et c.

Ensuite on construit les 6 autres sommets que l'on relie comme indiqué :



On peut remarquer que le sommet W doit pouvoir être colorié en vert si et seulement si a OU b est colorié en vert. En effet s'ils sont tous les deux en fushia, alors les sommets U et V devront être coloriés en vert et en noir,

et donc W sera colorié en fushia (et pas en vert). Au contraire si un des sommets, disons a, est colorié en vert, alors, on peut colorier U en fushia, V en noir et W en vert. De même pour les trois derniers sommets X, Y et Z qui codent $((a \text{ OU } b) \text{ OU } c)$. Donc la contrainte sera satisfiable si et seulement si Z pourra être colorié en vert. On veut le forcer à être vert et rendre ainsi ce bout de graphe 3-coloriable seulement quand la contrainte est satisfaisable : on relie Z à “N” et à “F”.

Normalement vous devriez avoir un graphe avec

$$3 + 2\text{nb_variables} + 9\text{nb_contraintes}$$

sommets. Faire tourner 3Col dessus devrait vous informer si les contraintes initiales sont satisfaisables ou non.